

UNIVERSIDAD DEL VALLE DE GUATEMALA
CC 3090– Ingeniería de Software I
Sección 11
Ing. Lynette García



Investigación automatización de pruebas funcionales

Sebastián García Bustamante - 22291
Olivier Viau - 23544
Juan Cruz - 23110
Luis Pedro Lira - 23669
Juan Francisco Martínez - 23617

Ciudad de Guatemala, 22 de agosto del 2025

1. Herramientas o frameworks existentes para pruebas automáticas

Dado que nuestro proyecto utiliza **Node.js (backend con Express y Prisma)** y **React Native (frontend móvil)**, investigamos frameworks relevantes en ambas áreas:

Backend (Node.js + Express + Prisma)

- **Jest:** Framework muy popular para pruebas en JavaScript y TypeScript. Soporta pruebas unitarias, de integración y regresión. Fácil configuración y gran comunidad.
- **Supertest:** Librería que se integra con Jest o Mocha para probar endpoints HTTP de manera directa, simulando peticiones y validando respuestas sin necesidad de levantar el servidor.
- **Mocha + Chai:** Opción alternativa. Mocha es un framework flexible para pruebas y Chai añade aserciones. Se usa mucho en entornos backend, aunque requiere mayor configuración.
- **AVA:** Framework minimalista que permite ejecutar pruebas en paralelo y con buen soporte para funciones asíncronas. Ideal para proyectos Node.js modernos.

Frontend (React Native)

- **Jest + React Native Testing Library:** Combinación recomendada. Jest hace la ejecución de pruebas, mientras que React Native Testing Library facilita simular interacciones de usuario en componentes.
- **Detox:** Framework especializado en pruebas end-to-end para aplicaciones móviles (iOS y Android). Permite automatizar escenarios reales como login, navegación, inputs y validaciones visuales.
- **Appium:** Más amplio que Detox, pues permite pruebas automáticas en aplicaciones móviles multiplataforma, incluyendo Android e iOS. Es robusto, aunque más complejo de configurar.

2. Herramientas seleccionadas para el proyecto

Como grupo decidimos trabajar con:

- **Backend: Jest + Supertest**
 - **Razones:**
 - Integración directa con Express.
 - Permite probar endpoints de forma simple simulando peticiones HTTP.

- No requiere levantar el servidor, lo que agiliza la ejecución.
- Gran comunidad, documentación robusta y soporte para TypeScript.
- **Frontend: Jest + React Native Testing Library**
 - **Razones:**
 - Permite probar componentes de React Native de manera realista.
 - Simula interacciones del usuario (inputs, botones, formularios).
 - Se integra fácilmente con Jest, que ya usamos en backend.
 - Popularidad en la industria y buena documentación.

3. Ejemplos de pruebas de integración automatizadas

Backend (3 ejemplos con Jest + Supertest)

1. Registro de usuario

- a. Probar que un usuario con datos válidos se registra exitosamente y devuelve código 201.

2. Login de usuario

- a. Verificar que al enviar credenciales correctas se recibe el mensaje *"Inicio de sesión exitoso"* y status 200.

3. Consulta de información del tutor

- a. Simular petición GET `/tutores/info/usuario/:id` y validar que la respuesta contenga nombre, materias y rating promedio.

Frontend (3 ejemplos con Jest + React Native Testing Library)

4. Pantalla de registro

- a. Validar que al dejar campos vacíos aparezca el mensaje *"Todos los campos son obligatorios"*.

5. Login

- a. Simular la escritura en inputs de correo y contraseña, presionar botón y verificar que se dispara la función `login()`.

6. Navegación hacia detalle de tutor

- a. Renderizar lista de tutores, simular click en un tutor y validar que se navega a la pantalla de detalle mostrando la información correspondiente.

4. Conclusiones

- **Eficiencia:** La combinación de Jest y Supertest en backend, y Jest con React Native Testing Library en frontend, permitió pruebas rápidas y sin configuraciones complicadas.
- **Tiempo empleado:** La implementación inicial tomó alrededor de 1 a 2 horas por herramienta, pero posteriormente las pruebas se ejecutan en segundos.
- **Calidad:** Las pruebas de integración nos permiten asegurar que tanto el backend como el frontend interactúan correctamente con los flujos principales del sistema (registro, login, consulta de tutorías).
- **Retos afrontados:** En backend, ajustar Supertest para simular correctamente los endpoints. En frontend, simular navegación fue un desafío, pero se resolvió integrando correctamente React Navigation en las pruebas.
- **Valor agregado:** La automatización de pruebas nos da confianza en que la evolución del sistema no romperá funcionalidades clave, mejorando la calidad del software entregado y la experiencia de usuario.

Evidencia de pruebas

[Link video de pruebas funcionales](#)

Referencias

Callstack. (s. f.). *React Native Testing Library*. Recuperado el 22 de agosto de 2025, de <https://callstack.github.io/react-native-testing-library/>

Detox. (s. f.). *Mobile end-to-end testing framework*. Wix. Recuperado el 22 de agosto de 2025, de <https://wix.github.io/Detox/>

Facebook. (s. f.). *Jest: Delightful JavaScript testing*. Recuperado el 22 de agosto de 2025, de <https://jestjs.io/docs/getting-started>

GeeksforGeeks. (2023, marzo 24). *Unit testing of Node.js application*. Recuperado de <https://www.geeksforgeeks.org/unit-testing-of-node-js-application/>

Mocha. (s. f.). *Mocha documentation*. Recuperado el 22 de agosto de 2025, de <https://mochajs.org/>

npm, Inc. (s. f.). *Supertest*. npm. Recuperado el 22 de agosto de 2025, de <https://www.npmjs.com/package/supertest>

npm, Inc. (s. f.). *AVA*. npm. Recuperado el 22 de agosto de 2025, de <https://www.npmjs.com/package/ava>

Testim. (2021, noviembre 9). *Supertest: How to test APIs like a pro*. Testim Blog. Recuperado de <https://www.testim.io/blog/supertest-how-to-test-apis-like-a-pro/>

Appium Foundation. (s. f.). *Appium: Cross-platform automation*. Recuperado el 22 de agosto de 2025, de <https://appium.io/docs/en/about-appium/intro/>